

الگوریتم‌های حریصانه (Greedy Algorithms)

- الگوریتم‌های حریصانه مانند برنامه نویسی پویا برای حل مسائل بهینه سازی استفاده می شوند
- در این روش در هر مرحله از بین انتخاب‌های ممکن گزینه ای را انتخاب می کند که در همان لحظه بهترین انتخاب به نظر می رسد. بدون توجه به انتخاب‌هایی که قبلاً انجام داده یا در آینده انجام خواهد داد
- با این روش نمی توان برای هر مسئله‌ای پاسخ بهینه را بدست آورد.

ایده : وقتی انتخابی دارید، گزینه‌ای را انتخاب کنید که در حال حاضر بهترین به نظر می‌رسد. یک انتخاب محلی بهینه انجام دهید به امید رسیدن به یک راه حل سراسری بهینه. الگوریتم‌های حریصانه همیشه راه حل بهینه را تولید نمی‌کنند. اما گاهی اوقات این کار را می‌کنند. ما مسئله‌ای را خواهیم دید که این اتفاق می‌افتد. سپس به برخی ویژگی‌های کلی، زمانی که الگوریتم‌های حریصانه راه حل‌های بهینه می‌دهند نگاه می‌کنیم. همچنین دو کاربرد دیگر روش حریصانه را مطالعه می‌کنیم: کدینگ هافمن و کشینگ آفلاین. [فصل‌های بعدی نیز از روش حریصانه استفاده می‌کنند: درخت پوشای کمینه، الگوریتم دایجسترا برای کوتاه‌ترین مسیرهای تک‌مبدأ، و یک هیوریستیک حریصانه پوشش مجموعه].

مثال : فرض کنید می خواهید یک سکه ۶۲ واحدی را با استفاده از سکه های ۱، ۵، ۱۰ و ۲۵ واحدی خرد کنید، بطوریکه که کمترین تعداد سکه مورد نیاز باشد.

راه حل حریصانه: استفاده از سکه با ارزش بیشتر برای خرد کردن است. یعنی برای مثال مطرح شده اول به سراغ سکه ۲۵ خواهیم رفت.

$$\begin{array}{ccccccc} \textcircled{25} & & \textcircled{10} & & \textcircled{5} & & \textcircled{1} \\ 2x \textcircled{25} + 1x \textcircled{10} & & \textcircled{5} & & 2x \textcircled{1} & \rightarrow & 5 \end{array}$$

(ب) حال فرض کنید سکه های ما به سکه های ۱، ۲، ۱۰، ۳۰ و ۳۵ واحدی تغییر کند:

راه حل حریصانه

$$\begin{array}{ccccccc} \textcircled{35} & & \textcircled{30} & & \textcircled{10} & & \textcircled{2} & & \textcircled{1} \\ 1x \textcircled{35} & & \textcircled{30} & + & 2x \textcircled{10} & + & 3x \textcircled{2} & + & 1x \textcircled{1} \rightarrow 7 \end{array}$$

اما یک راه حل بهتر وجود دارد:

$$\textcircled{35} \quad 2x \textcircled{30} \quad \textcircled{10} \quad + \quad 1x \textcircled{2} \quad \textcircled{1} \rightarrow 3$$

الگوریتم حریصانه همیشه بهترین جواب را پیدا نمی کند.

- پرداخت ۱۸ سنت با سکه های       حریصانه: دو سکه ۷ سنتی و چهار سکه ۱ سنتی
- بهینه: سه تا سکه ۶ سنتی 

- پرداخت ۱۶ سنت با سکه های ۱۲، ۱۰، ۵ و ۱ سنتی
- حریصانه: یک سکه ۱۲ سنتی و چهار سکه ۱ سنتی
- بهینه: یک سکه ۱۰ سنتی، یک سکه ۵ سنتی و یک سکه ۱ سنتی

الگوریتم های حریصانه دارای یک حلقه (چرخه) هستند. در هر دور تکرار الگوریتم حریصانه شامل مراحل زیر است:

- **انتخاب: حریصانه:** از بین عناصر، بهترین عنصر در نظر گرفته می شود. این انتخاب براساس یک معیار حریصانه که به طور محلی بهترین جواب را در هر لحظه انتخاب می کند شکل می گیرد.
- **بررسی امکان سنجی:** بررسی می شود که آیا با انتخاب آن مولفه امکان رسیدن به جواب وجود دارد یا خیر
 - در صورت مثبت بودن پاسخ، آن مولفه را به مجموعه جواب اضافه می کنیم
 - در صورت منفی بودن پاسخ، آن مولفه را برای همیشه کنار می گذاریم
- **بررسی جواب:** با افزودن هر عنصر به مجموعه جواب، بررسی می کنیم اگر جواب مسئله حاصل شده است. جواب به دست آمده کار تمام است

مثال: مسئله انتخاب فعالیت

n فعالیت نیاز به استفاده انحصاری از یک منبع مشترک دارند. برای مثال، زمان‌بندی استفاده از یک کلاس درس.

مجموعه فعالیت‌ها $S = \{a_1, \dots, a_n\}$ را به این صورت در نظر بگیرید

آن گاه یک فعالیت a_i در بازه $[s_i, f_i)$ به منبع نیاز دارد، که یک بازه نیمه‌باز است، جایی که

s_i : زمان شروع فعالیت i

f_i : زمان پایان فعالیت i

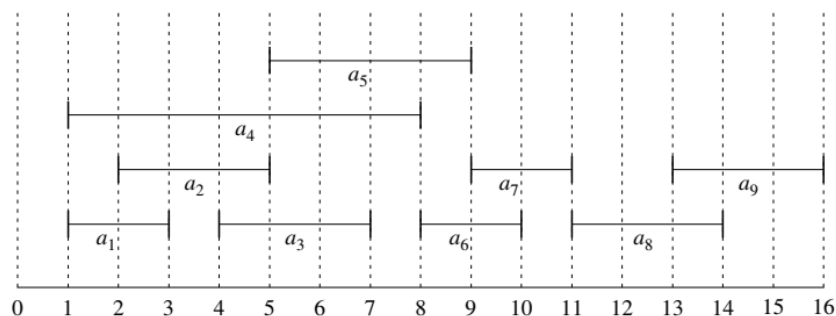
هدف: بزرگترین مجموعه ممکن از فعالیت‌های غیرهمپوشان (سازگار متقابل) را انتخاب کنید. انتخاب بیشترین تعداد فعالیت قابل انجام برای مثال:

- زمان‌بندی کلاس برای طولانی‌ترین زمان.
- بیشینه کردن درآمد اجاره‌ای.

فرض کنید فعالیت‌ها بر اساس زمان پایان مرتب شده‌اند: $f_1 \leq f_2 \leq f_3 \leq \dots \leq f_{n-1} \leq f_n$.

مثال S مرتب شده بر اساس زمان پایان:

i	1	2	3	4	5	6	7	8	9
s_i	1	2	4	1	5	8	9	11	13
f_i	3	5	7	8	9	10	11	14	16



بزرگترین مجموعه سازگار متقابل: $\{a_1, a_3, a_6, a_8\}$.

پاسخ در این مسئله منحصر بفرد نیست

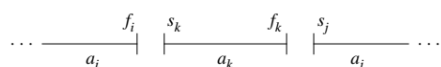
$\{a_1, a_3, a_6, a_9\}, \{a_1, a_3, a_7, a_8\}, \{a_1, a_3, a_7, a_9\}, \{a_1, a_5, a_7, a_8\}$

حل مسئله انتخاب فعالیت با استفاده از برنامه‌نویسی پویا

گام نخست برای حل مسئله با برنامه‌نویسی پویا این است که مسئله را به زیرمسئله‌های بهینه تقسیم کنیم.

یک زیرمجموعه از فعالیت‌ها را در نظر بگیرید، آن را با S_{ij} نشان می‌دهیم. این فعالیت‌ها، بعد از پایان فعالیت (وظیفه) a_i شروع شده و قبل از شروع فعالیت a_j پایان یابند.

$$S_{ij} = \{a_k \in S : f_i \leq s_k < f_k \leq s_j\}$$



فعالیت‌های موجود در S_{ij} با موارد زیر سازگار هستند (همپوشانی ندارند) :

- همه فعالیت‌هایی که تا f_i پایان می‌یابند، و
- همه فعالیت‌هایی که زودتر از s_j شروع نمی‌شوند.

فرض کنید A_{ij} یک مجموعه با بیشینه اندازه از فعالیت‌های سازگار متقابل در S_{ij} باشد.

همچنین فرض کنید $a_k \in A_{ij}$ یک فعالیت در A_{ij} باشد. سپس دو زیرمسئله داریم:

۱. یافتن فعالیت‌های سازگار متقابل در S_{ik} (فعالیت‌هایی که بعد از پایان a_i شروع می‌شوند و قبل از شروع a_k پایان می‌یابند).

۲. یافتن فعالیت‌های سازگار متقابل در S_{kj} (فعالیت‌هایی که بعد از پایان a_k شروع می‌شوند و قبل از شروع a_j پایان می‌یابند).

با پیدا کردن دو مجموعه S_{ik} و S_{kj} آن گاه می‌توانیم مجموعه‌های A_{ik} و A_{kj} را نیز پیدا کرد. پس می‌توان نوشت:

$$A_{ik} = A_{ij} \cap S_{ik} = \text{activities in } A_{ij} \text{ that finish before } a_k \text{ starts,}$$

$$A_{kj} = A_{ij} \cap S_{kj} = \text{activities in } A_{ij} \text{ that start after } a_k \text{ finishes.}$$

در نتیجه خواهیم داشت

$$A_{ij} = A_{ik} \cup \{a_k\} \cup A_{kj}$$

$$\Rightarrow |A_{ij}| = |A_{ik}| + |A_{kj}| + 1$$

بنابراین، می‌توان ادعا کرد که راه‌حل بهینه A_{ij} باید شامل راه‌حل‌های بهینه برای دو زیرمسئله S_{ik} و S_{kj} باشد.

تا این جا ما نشان دادیم که که جواب بهینه مسئله اصلی چگونه از جواب‌های بهینه زیرمسئله‌ها ساخته می‌شود. به عبارت دیگر یک زیرساخت جواب بهینه را توصیف کردیم. این گام نخست در برنامه‌نویسی پویا است. همچنین ما باید درستی توصیف خود را اثبات کنیم

ادعا را برای S_{kj} نشان می‌دهیم؛ اثبات برای S_{ik} متقارن است.

فرض کنید می‌توانستیم یک مجموعه A'_{kj} از فعالیت‌های سازگار متقابل در S_{kj} پیدا کنیم، جایی که $|A'_{kj}| > |A_{kj}|$. سپس از A'_{kj} به جای A_{kj} هنگام حل زیرمسئله برای S_{ij} استفاده کنید. اندازه مجموعه حاصل از فعالیت‌های سازگار متقابل $|A| = |A_{ik}| + |A_{kj}| + 1 > |A_{ik}| + |A'_{kj}| + 1 = |A|$ خواهد بود. این با فرض بهینه بودن A_{ij} در تناقض است.

بعد از اثبات درستی، باید یک رابطه بازگشتی برای جواب بدست آوریم.

یک راه‌حل بازگشتی

از آنجا که راه‌حل بهینه A_{ij} باید شامل راه‌حل‌های بهینه برای زیرمسئله‌های S_{ik} و S_{kj} باشد، می‌توان آن را با برنامه‌نویسی پویا حل کرد.

فرض کنید $c[i, j] =$ اندازه راه‌حل بهینه برای S_{ij} (یعنی حداکثر تعداد کارهای سازگار با یکدیگر در S_{ij}) باشد، پس می‌توان نوشت:

$$c[i, j] = c[i, k] + c[k, j] + 1.$$

اما ما که نمی‌دانیم کدام فعالیت را به عنوان a_k را انتخاب کنیم، بنابراین باید همه آن‌ها را امتحان کنیم:

$$c[i, j] = \begin{cases} 0 & \text{if } S_{ij} = \emptyset, \\ \max_{(i < k < j)} \{c[i, k] + c[k, j] + 1 : a_k \in S_{ij}\} & \text{if } S_{ij} \neq \emptyset. \end{cases}$$

با بدست آوردن رابطه بازگشتی، ما می‌توانیم یک الگوریتم بازگشتی ایجاد کرده و آن را یادداشت کنیم. یا می‌توان یک الگوریتم پایین به بالا ایجاد کرد و ورودی‌های جدول را پر کرد.

با توجه به فرمول بالا ما می‌توانیم از برنامه نویسی پویا استفاده کنیم اما در این حالت باید تمام حالات را بررسی کنیم ولی اگر از برنامه نویسی حریصانه استفاده کنیم به جای تمام حالات فقط حالت بهینه را انتخاب و بررسی می‌کنیم.

پس در ادامه، به یک روش حریصانه نگاه خواهیم کرد.

انتخاب حریصانه

قبل از حل زیرمسئله‌ها، یک فعالیت را برای افزودن به راه‌حل بهینه انتخاب کنید. برای مسئله انتخاب فعالیت، می‌توانیم فقط با در نظر گرفتن انتخاب حریصانه کار کنیم: فعالیتی که منبع را برای بیشترین فعالیت‌های دیگر در دسترس می‌گذارد.

سوال: کدام فعالیت منبع را برای بیشترین فعالیت‌های دیگر در دسترس می‌گذارد؟
پاسخ: اولین فعالیتی که پایان می‌یابد. (اگر بیش از یک فعالیت دارای کمترین زمان پایان باشد، می‌توان هر یک از آن‌ها را انتخاب کرد.)

از آنجا که فعالیت‌ها بر اساس زمان پایان مرتب شده‌اند، فقط فعالیت a_1 را انتخاب کنید.

این فقط یک زیرمسئله برای حل باقی می‌گذارد: یافتن یک مجموعه با بیشینه اندازه (بیشترین تعداد) از فعالیت‌های سازگار متقابل که بعد از پایان a_1 شروع می‌شوند. (نیازی نیست نگران فعالیت‌هایی باشید که قبل از شروع a_1 پایان می‌یابند، زیرا $f_1 < s_1$ و هیچ فعالیت a_i زمان پایان $f_i < f_1$ ندارد $\Rightarrow$ هیچ فعالیت a_i با $s_1 \leq f_i$ وجود ندارد.)

در حقیقت به جای آن که مسئله را به دو زیرمسئله تقسیم کنیم، ما مسئله را به یک زیرمسئله با اندازه کوچکتر کاهش دادیم. از آنجا که فقط یک زیرمسئله برای حل داریم، نمادگذاری را ساده می‌کنیم:

تقسیم به دو زیرمسئله $s_{ij} \rightarrow s_{ik} \text{ and } s_{kj}$

کاهش به یک زیرمسئله با اندازه کوچکتر $s_{ij} \rightarrow s_k$

$$S_k = \{a_i \in S : s_i \geq f_k\} = \text{activities that start after } a_k \text{ finishes.}$$

انتخاب حریصانه $a_1 \Leftarrow S_1$ به عنوان تنها زیرمسئله برای حل باقی می‌ماند. اگر a_1 انتخاب حریصانه باشد آن گاه S_1 تنها زیرمسئله ای است که باید حل شود [سوء استفاده جزئی از نمادگذاری: اشاره به S_k نه تنها به عنوان مجموعه‌ای از فعالیت‌ها بلکه به عنوان یک زیرمسئله متشکل از این فعالیت‌ها].

زیرساخت بهینه: اگر a_1 در یک راه‌حل بهینه باشد، آنگاه یک راه‌حل بهینه برای مسئله اصلی شامل a_1 به علاوه تمام فعالیت‌های در یک راه‌حل بهینه برای S_1 است. به عبارت دیگر اگر پاسخ بهینه باشد دو زیرمسئله حاصل شده (یعنی a_1 و پاسخ برای مجموعه S_1) هم بهینه هستند.

اما باید ثابت کنیم که a_1 (که بصورت حریصانه انتخاب شده است و نخستین کاری است که پایان می‌یابد) همیشه بخشی از برخی راه‌حل‌های بهینه است.

قضیه

اگر S_k غیرخالی باشد و a_m کمترین زمان پایان را در S_k داشته باشد، آنگاه a_m در برخی راه‌حل‌های بهینه گنجانده شده است.

اثبات

فرض کنید A_k یک راه حل بهینه برای S_k باشد، و a_j کمترین زمان پلایان را در بین تمام فعالیت‌های A_k داشته باشد. اگر $a_j = a_m$ ، کار تمام است. در غیر این صورت، فرض کنید $A'_k = A_k - \{a_j\} \cup \{a_m\}$ باشد (یعنی a_j را با a_m جایگزین کنیم) که A_k است اما با a_m جایگزین a_j شده است.

یک راه حل بهینه را در نظر می‌گیریم اگر زودترین کاری که پایان می‌یابد a_m باشد که قضیه ثابت شده است ولی در غیر این صورت می‌توانیم زودترین کار این مجموعه که پایان می‌یابد را با a_m جایگزین کنیم

ادعا

چون فعالیت‌های در A'_k مجزا هستند و همپوشانی ندارند پس A'_k هم یک جواب است.

اثبات ادعا

فعالیت‌های در A_k مجزا هستند، a_j اولین فعالیت در A_k است که پایان می‌یابد، و $f_m \leq f_j$. (ادعا)
از آنجا که $|A'_k| = |A_k|$ ، نتیجه می‌گیریم که A'_k یک راه حل بهینه برای S_k است، و شامل a_m است.

بنابراین، نیازی به قدرت کامل برنامه‌نویسی پویا نیست. نیازی به کار از پایین به بالا نیست.

در مقابل با یک رویکرد حریصانه، می‌توانیم فقط به طور مکرر فعالیتی را انتخاب کنیم که زودتر پایان می‌یابد، فقط فعالیت‌هایی را نگه داریم که با آن سازگار هستند، و این کار را تا زمانی که هیچ فعالیتی باقی نماند تکرار کنیم.

می‌توانیم از بالا به پایین کار کنیم: یک انتخاب انجام دهیم، سپس یک زیرمسئله حل کنیم. نیازی به حل زیرمسئله‌ها قبل از انجام انتخاب نیست.

الگوریتم حریصانه بازگشتی

زمان‌های شروع و پایان با آرایه‌های S و f نشان داده می‌شوند، جایی که فرض می‌شود f از قبل به صورت یکنوا افزایشی مرتب شده است.

برای شروع، یک فعالیت ساختگی a_0 با $f_0 = 0$ اضافه کنید، به طوری که $S_0 = S$ ، کل مجموعه فعالیت‌ها.

روال Recursive-Activity-Selector پارامترهای آرایه‌های S و f ، اندیس k زیرمسئله جاری، و تعداد n فعالیت‌ها در مسئله اصلی را می‌گیرد.

RECURSIVE-ACTIVITY-SELECTOR(s, f, k, n)

```
 $m = k + 1$   
while  $m \leq n$  and  $s[m] < f[k]$  // find the first activity in  $S_k$  to finish  
     $m = m + 1$   
if  $m \leq n$   
    return  $\{a_m\} \cup \text{RECURSIVE-ACTIVITY-SELECTOR}(s, f, m, n)$   
else return  $\emptyset$ 
```

فراخوانی اولیه

Recursive-Activity-Selector($s, f, 0, n$)

ایده

حلقه **while** $a_{k+1}, a_{k+2}, \dots, a_n$ را بررسی می‌کند تا زمانی که یک فعالیت a_m پیدا کند که با a_k سازگار است (نیاز به $s_m \geq f_k$ دارد).

- اگر حلقه به دلیل یافتن a_m پایان یابد ($m \leq n$)، سپس S_m را به صورت بازگشتی حل کنید و این راه‌حل را به همراه a_m برگردانید.

- اگر حلقه هیچ a_m فعالیت سازگاری (غیرهمپوشان) پیدا نکند ($m > n$)، فقط مجموعه خالی را برگردانید.

از مثال داده شده قبلی عبور کنید. باید $\{a_1, a_3, a_6, a_8\}$ را دریافت کنید.

زمان

$\Theta(n)$ — هر فعالیت دقیقاً یک بار بررسی می‌شود، با فرض اینکه فعالیت‌ها از قبل بر اساس زمان پایان مرتب شده‌اند.

الگوریتم حریصانه غیربازگشتی

می‌توان الگوریتم بازگشتی را به یک الگوریتم تکراری تبدیل کرد. در حال حاضر تقریباً بازگشتی است.

GREEDY-ACTIVITY-SELECTOR(s, f, n)

```
 $A = \{a_1\}$   
 $k = 1$   
for  $m = 2$  to  $n$   
    if  $s[m] \geq f[k]$  // is  $a_m$  in  $S_k$ ?  
         $A = A \cup \{a_m\}$  // yes, so choose it  
         $k = m$  // and continue from there  
return  $A$ 
```

مثال داده شده قبلی را دوباره اجرا کنید. باید دوباره $\{a_1, a_3, a_6, a_8\}$ را دریافت کنید.

زمان

$\Theta(n)$ ، اگر فعالیت‌ها از قبل بر اساس زمان پایان مرتب شده باشند. برای هر دو الگوریتم بازگشتی و تکراری (غیربازگشتی)،

اگر فعالیت‌ها نیاز به مرتب‌سازی داشته باشند، آنگاه $O(n \lg n)$ زمان اضافه کنید.

عناصر استراتژی حریصانه

انتخابی که در حال حاضر بهترین به نظر می‌رسد را انتخاب می‌کنیم.

برای مسئله انتخاب فعالیت چه کار کردیم؟

۱. تعیین زیرساخت بهینه

۲. توسعه یک راه‌حل بازگشتی

۳. نشان دادن اینکه اگر انتخاب حریصانه را انجام دهید، فقط یک زیرمسئله باقی می‌ماند

۴. اثبات اینکه همیشه، انتخاب حریصانه ایمن است

۵. توسعه یک الگوریتم حریصانه بازگشتی

۶. تبدیل آن به یک الگوریتم با حلقه چرخش

در ابتدا، شبیه برنامه‌نویسی پویا به نظر می‌رسید. در مسئله انتخاب فعالیت، با تعریف زیرمسئله‌های S_{ij} شروع کردیم، جایی که هر دو i و j متغیر بودند. اما سپس متوجه شدیم که انجام یک انتخاب حریصانه به ما اجازه می‌دهد زیرمسئله‌ها را به شکل S_k محدود کنیم.

می‌توانستیم مستقیماً به سراغ روش حریصانه برویم: در اولین تلاش برای تعریف زیرمسئله‌ها، از فرم S_k استفاده کنیم. سپس می‌توانستیم ثابت کنیم که انتخاب حریصانه a_m (اولین فعالیتی که پایان می‌یابد)، در ترکیب با راه‌حل بهینه برای فعالیت‌های سازگار باقی‌مانده S_m ، یک راه‌حل بهینه برای S_k می‌دهد.

به طور معمول، مراحل را بصورت زیر ساده می‌کنیم:

۱. مسئله بهینه‌سازی را به گونه‌ای مطرح کنید که شامل یک انتخاب و یک زیرمسئله که باید شود، باشد

۲. ثابت کنید که همیشه با انجام انتخاب حریصانه یک راه‌حل بهینه وجود دارد، بنابراین انتخاب حریصانه همیشه ایمن است

۳. نشان دادن زیرساخت بهینه به وسیله این که اینکه پس از انتخاب حریصانه، ترکیب یک راه‌حل بهینه برای زیرمسئله

باقیمانده با انتخاب حریصانه، یک راه‌حل بهینه برای مسئله اصلی ارائه می‌دهد.

هیچ روش کلی برای تشخیص اینکه آیا یک الگوریتم حریصانه بهینه است وجود ندارد، اما دو عنصر کلیدی وجود دارد

- ویژگی انتخاب حریصانه
- زیرساخت بهینه